

Chapter Overview

Chapter 16 covers Docker containerisation for ShopVerse: image layering, multi-stage builds, Dockerfile best practices, Docker Compose for local dev, BuildKit, image security scanning, registry management, networking, volumes, health checks, running Spring Boot optimally in containers, and container resource limits.

Sub-Chapter	Topic
16-01	Docker Architecture – daemon, images, containers, registries
16-02	Image Layering – Union FS, layer caching, base image selection
16-03	Dockerfile Best Practices – ShopVerse multi-stage build
16-04	Multi-Stage Build – builder + runtime stages
16-05	BuildKit – parallel builds, cache mounts, secret mounts
16-06	Docker Compose – local dev stack (all ShopVerse services)
16-07	Docker Networking – bridge, host, overlay networks
16-08	Volumes & Bind Mounts – data persistence strategies
16-09	Spring Boot in Docker – JVM flags, memory limits, layered JAR
16-10	Health Checks – HEALTHCHECK instruction, Compose depends_on
16-11	Image Security – non-root user, secret mounts, scanning
16-12	Container Registry – Docker Hub, ECR, image tagging strategy
16-13	Resource Limits & cgroups – CPU, memory, OOM killer

Docker Architecture

Docker Client (docker CLI) – sends commands to daemon via REST API

docker build, docker run, docker push, docker pull, docker compose

Docker Daemon (dockerd) – manages containers, images, volumes, networks

Listens on /var/run/docker.sock; can also be remote TCP

Container Runtime – containerd (default OCI runtime)

runc: actually forks/execs container process using Linux primitives

Image Registry – stores and distributes images

Docker Hub, AWS ECR, GitHub GHCR, Harbor (private/on-prem)

Container – running instance of an image

Isolated via Linux namespaces: PID, NET, MNT, UTS, IPC, User

Resource-limited via cgroups v2: CPU shares, memory limit, I/O

Concept	VM	Container
Isolation	Full OS virtualisation (hypervisor)	Linux namespaces + cgroups (kernel shared)
Boot time	Minutes (full OS boot)	Milliseconds (process fork)
Image size	GBs (full OS + app)	MBs (app + minimal OS libs)
Overhead	High (guest OS memory)	Minimal – kernel shared with host
Use for	Strong multi-tenant isolation	App packaging, CI, microservices

Union FS – How Layers Stack

Docker Image: shopverse/product-service:1.0 (overlay2 Union FS)

Layer 1: eclipse-temurin:21-jre-alpine base OS + JRE – 85 MB – CACHED most

Layer 2: RUN addgroup + adduser (non-root user setup) – tiny – CACHED

Layer 3: COPY dependencies/ (3rd-party JARs from Maven Central) – CACHED if

Layer 4: COPY spring-boot-loader/ (Spring Boot loader classes) – CACHED

Layer 5: COPY snapshot-dependencies/ (internal SNAPSHOT JARs) – CACHED

Layer 6: COPY application/ (YOUR code .class files) – REBUILT every commit

Layer	Content	Changes How Often	Strategy
Base OS + JRE	eclipse-temurin:21-jre-alpine	Monthly (security patches)	Cache digest for reproducibility
OS-level tools	tzdata, curl (via RUN apk add)	Rarely	Combine all RUN apk add in one layer
3rd-party JARs	Spring Boot, Hibernate, Jackson	Weekly at most	Separate layer; cache on pom.xml unchanged
Application code	Your .class files, resources	Every commit	Last layer – only this rebuilds per commit

Rule: put rarely-changing instructions first. Only layers after a change are invalidated. With layered JAR, a typical commit rebuilds only the application layer (~1 MB) not the 3rd-party deps (~60 MB).

16-03 & 16-04 Dockerfile – Multi-Stage Build

```
# ---- Stage 1: Build (JDK) ----
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /build
COPY pom.xml . \.mvn .mvn mvnw .
RUN --mount=type=cache,target=/root/.m2 \
  ./mvnw dependency:go-offline -q
COPY src ./src
RUN --mount=type=cache,target=/root/.m2 \
  ./mvnw package -DskipTests -q
RUN java -Djarmode=layertools -jar target/*.jar extract --destination /app
# ---- Stage 2: Runtime (JRE only) ----
FROM eclipse-temurin:21-jre-alpine AS runtime
RUN addgroup -S shopverse && adduser -S app -G shopverse
WORKDIR /app
COPY --from=builder /app/dependencies/ ./dependencies/
COPY --from=builder /app/spring-boot-loader/ ./spring-boot-loader/
COPY --from=builder /app/snapshot-dependencies/ ./snapshot-dependencies/
COPY --from=builder /app/application/ ./application/
USER app
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=5s --start-period=60s --retries=3 \
  CMD wget -qO- http://localhost:8080/actuator/health | grep -q UP || exit 1
ENTRYPOINT ["java",
  "-XX:+UseZGC",
  "-XX:MaxRAMPercentage=75.0",
  "-Djava.security.egd=file:/dev/./urandom",
  "org.springframework.boot.loader.launch.JarLauncher"]
```

Stage	Base Image	Size	Purpose
builder	eclipse-temurin:21-jdk-alpine	~300 MB	Full JDK; Maven compile + package
runtime	eclipse-temurin:21-jre-alpine	~85 MB	JRE only; no compiler; minimal attack surface

```
# Enable BuildKit (required for --mount syntax)
DOCKER_BUILDKIT=1 docker build .
# Or permanently in /etc/docker/daemon.json:
{ "features": { "buildkit": true } }
# Cache mount – Maven local repo persists across builds
RUN --mount=type=cache,target=/root/.m2 ./mvnw package -DskipTests
# Result: deps NOT re-downloaded on each build – 5-10× faster CI
# Secret mount – API key at build time; NOT baked into any layer
RUN --mount=type=secret,id=github_token \
GITHUB_TOKEN=$(cat /run/secrets/github_token) ./fetch-private-artifact.sh
# Invoke with:
docker build --secret id=github_token,src=~/.github_token .
# SSH mount – forward SSH agent for private Git repos
RUN --mount=type=ssh git clone git@github.com:shopverse/internal-lib.git
```

BuildKit Feature	Benefit	Before BuildKit
--mount=type=cache	Persist Maven/npm cache across builds	Cached in Docker every build
--mount=type=secret	Pass secrets without baking into image	ENV SECRET=... visible in docker history
--mount=type=ssh	Use host SSH agent for git clone	SSH key COPY into image (security risk)
Parallel FROM stages	Multi-stage stages run concurrently	Sequential; slower total build time

```
# docker-compose.yml - ShopVerse complete local stack
services:
  postgres:
    image: postgres:16-alpine
    environment: { POSTGRES_DB: shopverse, POSTGRES_PASSWORD: shopverse }
    ports: ["5432:5432"]
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL","pg_isready -U postgres"]
      interval: 10s timeout: 5s retries: 5
  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
    volumes: [redisdata:/data]
  kafka:
    image: confluentinc/cp-kafka:7.6
    environment:
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
    ports: ["9092:9092"]
  product-service:
    build: { context: ./product-service }
    ports: ["8080:8080"]
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/shopverse
      SPRING_REDIS_HOST: redis
      SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:9092
    depends_on:
      postgres: { condition: service_healthy }
    volumes: { pgdata: , redisdata: }
```

Network Type	Scope	Use Case	ShopVerse Use
bridge (default)	Single host	Container-to-container on one Docker host	Local Compose dev stack
host	Single host	Share host network; no NAT	Overhead Compose; avoid isolation loss
overlay	Multi-host	Cross-host container network	K8s (Swave/K8s/Calico)
none	Isolated	No networking at all	Offline batch computation

```
# Custom bridge network - containers resolve each other by service name
networks:
shopverse-net:
driver: bridge
ipam: { config: [{ subnet: 172.28.0.0/16 }] }
services:
postgres: { networks: [shopverse-net] }
product-service:
networks: [shopverse-net]
# reaches postgres via hostname "postgres" port 5432
```

DNS	How	Example
Compose service name	Resolves to container IP with postgres	postgres:5432 from product-service
K8s Service DNS	ClusterDNS resolves <svc> in cluster	inventory-service:8082

Type	Syntax	Managed By	Use Case
Named Volume	volumes: pgdata:/var/lib/postgresql/data	Docker	DB persistence; survives container removal
Anonymous Volume	VOLUME /app/logs in Dockerfile	Docker	Ephemeral; created per container; not for persistence
Bind Mount	- ./src:/app/src:ro	Host OS	Dev: mount source live into container for hot reload

```
# Production: named volumes for all stateful services
```

```
volumes:
```

```
pgdata:
```

```
esdata:
```

```
redisdata:
```

```
# Development only: bind mount for live code reload
```

```
product-service:
```

```
volumes:
```

```
- ./product-service/src:/app/src:ro
```

```
- ./product-service/target:/app/target:ro
```

```
# Volume backup (named volume → tar)
```

```
docker run --rm -v pgdata:/data -v $(pwd):/backup alpine \
```

```
tar czf /backup/pgdata-backup.tar.gz -C /data .
```


JVM Flag	Value	Purpose
-XX:+UseZGC	—	Low-pause GC; < 1ms pauses; ideal for latency-sensitive containers
-XX:MaxRAMPercentage	75.0	Use 75% of container memory cgroup limit for heap
-XX:+UseContainerSupport	default Java 11+	JVM reads cgroup CPU/memory limits, not host specs
-Djava.security.egd	file:/dev/./urandom	Non-blocking entropy source; prevents 30s startup delays
-XX:+HeapDumpOnOutOfMemoryError		Dump heap to /tmp on OOM for post-mortem analysis

```
# Kubernetes Pod resource limits + JVM relationship
```

```
resources:
```

```
requests: { memory: "512Mi", cpu: "250m" }
```

```
limits: { memory: "1Gi", cpu: "1000m" }
```

```
# -XX:MaxRAMPercentage=75.0 → JVM heap = 768 MB (75% of 1 GiB limit)
```

```
# Without UseContainerSupport (old Java 8): reads host 64 GB → allocates 16 GB → OOM killed
```

```
# Layered JAR startup: extract once, COPY layers separately in Dockerfile
```

```
# Result: Spring Boot startup time reduces from 12s to 8s (pre-extracted layers)
```

```
# Dockerfile HEALTHCHECK instruction
HEALTHCHECK --interval=30s --timeout=5s --start-period=60s --retries=3 \
CMD wget -qO- http://localhost:8080/actuator/health | grep -q '"status":"UP"' || exit 1
# Compose: conditional startup ordering
product-service:
  depends_on:
    postgres: { condition: service_healthy }
    redis: { condition: service_started }
```

Parameter	Value	Why
--start-period	60s	Spring Boot takes 30-60s to start; no failures counted in this window
--interval	30s	Check every 30s during normal operation
--timeout	5s	Kill check if wget hangs longer; indicates overloaded container
--retries	3	3 consecutive failures = container marked unhealthy

Actuator Endpoint	Status	Consumer
/actuator/health	Overall UP/DOWN (all indicators)	Docker HEALTHCHECK
/actuator/health/liveness	JVM alive (not deadlocked)	K8s liveness probe – restart if fails
/actuator/health/readiness	Ready to serve traffic (DB connected)	K8s readiness probe – remove from Service

Practice	Implementation	Risk Prevented
Non-root user	USER app after adduser -S app	Root exploit in container → root on host
Minimal base image	eclipse-temurin:21-jre-alpine (85MB)	85MB packages = smaller CVE surface
No secrets in layers	--mount=type=secret; env vars	docker history reveals baked secrets
Read-only filesystem	--read-only at docker run / K8s securityContext	Prevents Denial of Service exploits
Image scanning	Trivy, Docker Scout, Snyk in CI	Detect CVEs in base image + dependencies
Pin base image digest	FROM eclipse-temurin@sha256:...	Reproducible; no surprise upstream change

```
# Trivy scan in GitHub Actions CI
- name: Scan image for CVEs
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: ghcr.io/shopverse/product-service:${{ github.sha }}
    severity: CRITICAL,HIGH
    exit-code: 1 # block deployment on CRITICAL/HIGH CVE
```

Tag Strategy	Example	Use When
Immutable SHA tag	product-service:abc1234	Every CI build; always use in K8s deployments
Semantic version	product-service:1.4.2	Release artifacts; changelog anchored
Environment tag (immutable)	product-service:prod-latest	Points to current production image
latest (mutable)	product-service:latest	Local dev only; NEVER in production K8s

```
# CI tagging pipeline
IMAGE=ghcr.io/shopverse/product-service
SHA=$(git rev-parse --short HEAD)
docker build -t ${IMAGE}:${SHA} -t ${IMAGE}:${cat VERSION} .
docker push ${IMAGE}:${SHA}
docker push ${IMAGE}:${cat VERSION}
# Kubernetes deployment always uses SHA:
image: ghcr.io/shopverse/product-service:abc1234
imagePullPolicy: IfNotPresent # safe: SHA = immutable
```

Registry	Auth	Best For
Docker Hub	docker login	Public images; OSS projects
AWS ECR	aws ecr get-login-password	AWS EKS workloads
GitHub GHCR	GITHUB_TOKEN	GitHub Actions CI; free for public repos
Harbor	Self-hosted	Air-gapped; enterprise policy; image replication

cgroup Control	docker run Flag	K8s Manifest Field	Effect
Memory limit	--memory=1g	resources.limits.memory: 1000Mi	OOM killer kills process if exceeded
Memory request	—	resources.requests.memory: 512Mi	Scheduler places pod on node with 512Mi free
CPU limit	--cpus=1.0	resources.limits.cpu: 1000m	Container cannot use more than 1 CPU core
CPU request	—	resources.requests.cpu: 250m	Guaranteed 0.25 CPU; schedule guarantee
OOM score	—	Set automatically by K8s QoS class	BestEffort killed first; Guaranteed last

```
# Kubernetes QoS classes
```

```
# Guaranteed: requests == limits for all containers → never OOM killed first
```

```
resources:
```

```
requests: { memory: "1Gi", cpu: "500m" }
```

```
limits: { memory: "1Gi", cpu: "500m" }
```

```
# Burstable: requests < limits → may OOM kill under node pressure
```

```
resources:
```

```
requests: { memory: "512Mi", cpu: "250m" }
```

```
limits: { memory: "1Gi", cpu: "1000m" }
```

```
# BestEffort: no requests or limits → killed first under pressure; never use for prod
```

QoS Class	How to Achieve	OOM Kill Priority
Guaranteed	requests == limits (all containers)	Last to be killed
Burstable	requests < limits or partial settings	Middle priority
BestEffort	No requests or limits at all	First to be killed

■ Do

- ✓ Use multi-stage builds: JDK builder → JRE-only runtime; reduces image 70%.
- ✓ Order Dockerfile layers by change frequency – base OS → deps → code (most volatile last).
- ✓ Always run as non-root USER app in containers – root in container = root on host without user NS.
- ✓ Set -XX:MaxRAMPercentage=75.0 not -Xmx for containers – reads cgroup limit dynamically.
- ✓ Use BuildKit --mount=type=cache for Maven/npm – dramatically faster CI incremental builds.
- ✓ Tag images with git SHA (immutable) in CI and production K8s deployments.
- ✓ Scan images with Trivy in CI; fail pipeline on CRITICAL/HIGH CVEs before deployment.
- ✓ Set HEALTHCHECK with start-period=60s – prevents Spring Boot startup from triggering restart loop.
- ✓ Use named Docker volumes for database data – portable; survives container removal.
- ✓ Set K8s resources.requests == resources.limits (Guaranteed QoS) for production services.

■ Anti-Patterns

- Running as root in containers – root exploit in container = root on host without user namespaces.
- Baking secrets (ENV SECRET=xyz) into Dockerfile – visible in docker history after build.
- Using latest tag in Kubernetes – unpredictable; new push changes what K8s pulls next restart.
- Single-stage build with full JDK in runtime image – 300 MB instead of 85 MB JRE.
- No HEALTHCHECK – Docker/K8s cannot detect a deadlocked Spring Boot application.
- COPY . . before dependency download – any code change invalidates the dependency cache layer.
- No resource limits in K8s – one misbehaving pod can starve all others on the node.

Sub-Chapter	Key Point	One-Liner
16-01 Architecture	Docker + runtime + registry	containerd + runc; namespaces + cgroups
16-02 Layering	volatile layers last	App code last; deps layer cached on pom unchanged
16-03/04 Multi-Stage	JDK builder → JRE runtime	Final image = 85 MB; JDK never ships
16-05 BuildKit	kit-mount=type=cache	Maven .m2 persists; secrets not baked in layers
16-06 Compose	depends_on + healthcheck	Postgres healthy before product-service starts
16-07 Networking	Bridge = service-name DNS	Postgres:5432 reachable by name in Compose
16-08 Volumes	Named volumes for DB	Bind mounts for dev hot-reload only
16-09 Spring JMX	MaxRAMPercentage=75	UseContainerSupport: reads cgroup not host RAM
16-10 Health	start-period=60s	Spring Boot startup time; avoid restart loop
16-11 Security	Non-root + Trivy CI scan	--mount=type=secret; pin base image digest
16-12 Registry	SHA tag everywhere	latest only in local dev; SHA in K8s
16-13 Resource requests	requests == limits = Guaranteed	BestEffort = first OOM killed

Interview Quick-Check

Question	Concise Answer
Multi-stage build benefit?	Stage 1: JDK to compile. Stage 2: JRE-only, copies artifacts. Final = 85 MB JRE runtime; 300 MB JDK
Layer caching strategy?	Rarely-changing layers first (base OS, 3rd-party JARs). App code last. Only layers after a change re
Why MaxRAMPercentage not -Xmx?	Container limit varies (HPA scaling, different envs). MaxRAMPercentage=75 dynamically uses 75%
HEALTHCHECK start-period?	Spring Boot takes 30-60s to start. Without start-period, health check fails during startup → Docker re
Guaranteed vs Burstable QoS?	Guaranteed: requests==limits; last OOM killed. Burstable: requests < limits; may OOM under pressu